

useState Hook

1. The Fundamental Problem

Question: How do you make a webpage interactive?

Answer: When data changes, the UI must change.

Example:

- User clicks a button → Counter should increase
 - User types in input → Text should appear on screen
 - User toggles a switch → Theme should change
-

2. Why Normal Variables Don't Work

```
function Counter() {  
  let count = 0; // Normal variable  
  
  return (  
    <button onClick={() => {  
      count = count + 1;  
      console.log(count); // Logs: 1, 2, 3...  
    }}>  
      Count: {count} {/* Always shows 0 on screen! */}  
    </button>  
  );  
}
```

What happens:

1. Click button → `count` becomes 1 in memory
2. Console shows `1` 

3. But UI still shows 0 ❌

Why?

- React doesn't know the variable changed
- Component function ran once, returned JSX, done
- To update UI, the component must **run again** (re-render)
- Normal variables can't trigger re-renders

3. What useState Does

`useState` gives you two things:

```
const [count, setCount] = useState(0);  
//      ↑       ↑           ↑  
//   value updater initial value
```

1. `count` - Current value of the state
2. `setCount` - Function to update the value AND trigger re-render
3. `useState(0)` - Sets initial value to 0

4. How useState Works (Step by Step)

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <button onClick={() => setCount(count + 1)}>  
      Count: {count}  
    </button>  
  );  
}
```

First Render:

1. `useState(0)` runs → React stores `0` internally
2. Returns `[0, setCount]`
3. `count = 0`
4. JSX: `<button>Count: 0</button>`
5. React renders button showing "Count: 0"

User Clicks Button:

1. `setCount(count + 1)` called → `setCount(1)`
2. React updates stored value from `0` to `1`
3. React schedules a re-render
4. **Component function runs again** (entire function from top)
5. `useState(0)` runs again, but this time returns `[1, setCount]`
6. `count = 1`
7. JSX: `<button>Count: 1</button>`
8. React updates DOM
9. Button now shows "Count: 1"

Key insight: The component function runs again, but `useState` remembers the updated value.

5. Where Does React Store State?

```
// Simplified: What React does internally

let stateStorage = []; // React keeps state here (outside your component)
let currentIndex = 0;

function useState(initialValue) {
  const index = currentIndex;
```

```

// First time: initialize
if (stateStorage[index] === undefined) {
  stateStorage[index] = initialValue;
}

const setState = (newValue) => {
  stateStorage[index] = newValue; // Update stored value
  reRenderComponent(); // Tell React to re-render
};

currentIndex++;
return [stateStorage[index], setState];
}

```

Key points:

- State lives **outside** your component function
- State **persists** between renders
- Each component instance has **its own state**

6. Re-render Means Running the Whole Function Again

```

function Counter() {
  console.log("Component function running!");

  const [count, setCount] = useState(0);

  const double = count * 2; // This is recalculated every re
  render

  return (
    <div>
      <p>Count: {count}</p>
      <p>Double: {double}</p>
    </div>
  );
}

```

```

    <button onClick={() => setCount(count + 1)}>Increment</button>
  </div>
);
}

```

Every click:

1. `setCount` called
2. Entire `Counter()` function runs again
3. Console logs "Component function running!"
4. `double` is recalculated
5. New JSX is created
6. React updates only what changed in the DOM

Important: All variables inside the function are **recreated** every render. Only state values persist (because React stores them externally).

7. State Updates Are NOT Immediate

```

function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    console.log("Before:", count); // 0

    setCount(count + 1);

    console.log("After:", count); // Still 0! (not 1)
  }

  return <button onClick={handleClick}>Count: {count}</button>
}

```

```
>;  
}
```

Why `count` is still 0 after `setCount` ?

- `setCount` **schedules** an update, doesn't execute immediately
- Your function must finish first
- Then React processes the update and re-renders
- In the next render, `count` will be 1

State updates are asynchronous!

8. Multiple `setState` Calls - Common Mistake

 **Wrong Way:**

```
function handleClick() {  
  setCount(count + 1); // setCount(0 + 1) = setCount(1)  
  setCount(count + 1); // setCount(0 + 1) = setCount(1)  
  setCount(count + 1); // setCount(0 + 1) = setCount(1)  
}  
// Result: count becomes 1 (not 3!)
```

Why? During the function, `count` is still 0. All three calls are `setCount(1)`.

 **Correct Way (Updater Function):**

```
function handleClick() {  
  setCount(prevCount => prevCount + 1); // 0 + 1 = 1  
  setCount(prevCount => prevCount + 1); // 1 + 1 = 2  
  setCount(prevCount => prevCount + 1); // 2 + 1 = 3  
}  
// Result: count becomes 3 
```

Rule: When new state depends on old state, use updater function.

9. Different Types of State

Numbers:

```
const [count, setCount] = useState(0);
setCount(5);
setCount(count + 1);
```

Strings:

```
const [name, setName] = useState('');
setName('John');
```

Booleans:

```
const [isOpen, setIsOpen] = useState(false);
setIsOpen(true);
setIsOpen(!isOpen); // Toggle
```

Arrays:

```
const [items, setItems] = useState([]);

// Add item
setItems([...items, newItem]);

// Remove item
setItems(items.filter(item => item.id !== idToRemove));

// Update item
setItems(items.map(item =>
  item.id === idToUpdate ? { ...item, name: 'New Name' } : it
```

```
em
));
```

Objects:

```
const [user, setUser] = useState({ name: '', age: 0 });

// Update one property
setUser({ ...user, name: 'John' });

// Update multiple properties
setUser({ ...user, name: 'John', age: 25 });
```

10. Multiple State Variables

```
function Form() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [age, setAge] = useState(0);

  return (
    <div>
      <input value={name} onChange={e => setName(e.target.value)} />
      <input value={email} onChange={e => setEmail(e.target.value)} />
      <input value={age} onChange={e => setAge(e.target.value)} />
    </div>
  );
}
```

You can have as many state variables as you need!

count --> Value Change
React ko bol du, ya signal de du

count use ho rha hai, udhr usko update kar do

